

Build Reality MVP as an end-to-end mobile-first PWA hosted on Vercel.

Product idea:

Reality MVP turns a rough product sketch or photo plus a text prompt into a spatial prototype. The user opens the app on their phone, captures or uploads a sketch, describes the product, sees a generated/selected 3D model, then places it in the real world using web AR.

Core demo flow:

1. User opens the Vercel-hosted PWA on mobile.
2. User captures/uploads a sketch using phone camera.
3. User enters a product prompt.
Example: "A smart water bottle for gym users that glows when hydration is low."
4. App analyzes the sketch and prompt.
5. App creates a product spec:
 - product name
 - product category
 - shape
 - materials
 - key features
 - intended use
 - refined 3D generation prompt
6. App selects a fallback .glb model or uses generated 3D model if available.
7. User sees a 3D preview.
8. User taps "View in AR."
9. The product opens in browser-based AR through <model-viewer>.
10. User can view the Codex Build Pack showing generated app files, product config, AGENTS.md, MVP_SPEC.md, README, and validation plan.

Technical requirements:

- Use Next.js App Router.
- Use TypeScript.
- Use Tailwind CSS.
- Host/deploy on Vercel.
- Make the UI mobile-first but usable on desktop.
- Make it installable as a PWA with manifest metadata.
- Use phone camera upload with:
`<input type="file" accept="image/*" capture="environment" />`
- Use <model-viewer> for 3D and AR preview.
- AR modes:
webxr scene-viewer quick-look
- Use fallback GLB models so the demo works even if external 3D generation fails.
- Store prototype data in localStorage or simple JSON state. No auth. No database.

Routes:

- /

Mobile-first create screen with camera upload, prompt input, example chips, and Generate Reality MVP button.

- /result/[id]

Shows product analysis, selected/generated 3D model, 3D preview, AR launch button, and build-pack link.

- /ar/[id]

Minimal mobile AR page with <model-viewer>, product name, feature callouts, and View in AR button.

- /build-pack/[id]

Shows Codex-generated software layer:

- app/ar/[id]/page.tsx

- product.config.json

- AGENTS.md

- MVP_SPEC.md

- VALIDATION_PLAN.md

- README section

Fallback models:

Place models in:

- public/models/bottle.glb

- public/models/lamp.glb

- public/models/chair.glb

- public/models/box.glb

- public/models/device.glb

If real 3D generation is not connected, classify the product category from the prompt and choose the closest fallback model.

Design direction:

- Clean, modern builder-tool UI.

- Mobile-first.

- No landing-page fluff.

- First screen should be the actual creation tool.

- Use compact cards, clear buttons, upload area, progress states, and model preview.

- Palette:

- background: #F8FAFC

- text: #0F172A

- primary: #2563EB

- success: #10B981

- warning/fallback: #F59E0B

- panels: #FFFFFF

- Border radius: 8px.

- Avoid heavy gradients and decorative blobs.

Demo data:

Default example should be:

Product: Smart Hydration Bottle

Prompt: "A smart water bottle for gym users that glows when hydration is low."

Category: bottle

Features:

- LED hydration reminder
- gym-friendly grip
- refill tracking
- companion app concept

Fallback model: /models/bottle.glb

Codex theme framing:

Do not pitch this as just AI 3D generation.

The core message is:

“Reality MVP uses Codex to generate the runnable spatial prototype app layer around a product concept.”

The Build Pack page must make this explicit:

- Codex creates the AR page
- Codex creates product config
- Codex creates AGENTS.md
- Codex creates MVP spec
- Codex creates validation plan
- Codex creates README submission content

Implementation priority:

1. Mobile-first PWA shell
2. Camera upload + prompt input
3. Mock/product analysis output
4. Fallback model selector
5. <model-viewer> AR page
6. Result page
7. Build Pack page
8. Vercel deployment readiness
9. Optional real OpenAI/Meshy integration

Do not build:

- native mobile app
- user accounts
- database
- custom camera streaming
- complex 3D editor
- multi-user collaboration

Final output should be a working Next.js project ready to deploy on Vercel.

Luke is typing

Compose

Write to OpenAI Hackathon